

# 受治理 Action 层 — 示例设计讨论全记录



一场从"本体论之辨"出发、落在一个贷款审批示例上的讨论:Palantir 的骨架 × 《决策原生方法论》的制度观 × Semantica 的载体。

文档按讨论的概念递进重排:背景 → 同构 → 概念 → 示例 → 生态 → 灵魂 → 边界 → 结论。蓝=Palantir 工程,橙=PDF 制度,青=合成/落地。

## 目录

- 1.0 [导言:这场讨论在回答什么](#)
- 2.1 [背景一:两种本体论](#)
- 3.2 [背景二:PDF 是 Palantir 的组织学翻译](#)
- 4.3 [核心概念:Action — 本体世界的动词](#)
- 5.4 [示例:「提高信用额度」完整走查](#)
- 6.5 [扩展:完整的 Action 生态](#)
- 7.6 [灵魂:PDF 的制度观\(五命题\)](#)
- 8.7 [现实边界:已有 vs 新设计](#)
- 9.8 [结论:对 fork 的意义](#)

## 0 导言:这场讨论在回答什么

讨论由一个追问链驱动:"Semantica 自称开源 Palantir,那 Palantir 的本体论到底是什么?" → "PDF 和它什么关系?" → "Action 怎么理解?" → "示例里的 Action 是项目已有的吗?" → "示例按谁的理论设计?" → "PDF 的制度观指什么?" 本文档保留这条逻辑线,整理为可复查的设计记录。

## 合成 三方嫁接 — 一句话总纲

### Palantir 本体论(骨架)

Objects · Links · Actions · Functions; Action 生命周期、风险分级审批、  
审计连锁

×  
注魂

### 《决策原生方法论》(魂)

判断/采纳三分 · 责任主体 · 强制 HITL · 不可变账本 · 知识单元版本化

↓  
落载体

### Semantica(fork,落地)

A-7291 贷款示例 · 已有的影子类(ApprovalChain / PolicyException /  
Policy.version) · 待建的 Action 引擎

■ Palantir 工程概念   ■ PDF 制度概念   ■ 合成 / 本项目落地

## 1 背景一:两种本体论

"本体"一词在两个世界里几乎是两种东西:**W3C 系**是知识表征的形式逻辑层,**Palantir** 是驱动运营的决策基质。区别不在"有没有本体",而在本体用来干什么。

Palantir 与 W3C 传统本体的对照

| 维度     | W3C / 传统本体                   | Palantir Ontology                         |
|--------|------------------------------|-------------------------------------------|
| 本质定位   | 知识表征:形式化概念体系                 | 运营语义层:组织的"数字孪生"                           |
| 形式基础   | OWL/RDF/SHACL/SKOS,描述逻辑,开放世界 | 刻意不用 OWL:强类型对象模型,务实建模                     |
| 核心构件   | 类、公理、层级、推理规则                 | Objects + Links + Actions + Functions 四件套 |
| "动词"在哪 | 没有动词——修改靠 SPARQL UPDATE      | Action 一等公民:带校验、审批、事务、审计的写回               |
| 目的     | 互操作、推理、知识共享                  | 决策——本体存在的意义是让"行动"可被治理地发生                  |
| 数据绑定   | 图即数据                         | 对象由数据管道物化,有血缘                             |
| 治理方式   | SHACL 校验 + PROV 溯源           | Action 审批流 + 权限标记 + 全程审计                  |
| 消费方式   | SPARQL 查询、推理机                | 应用、Agent(AIP 经同一套 Action API 行动)          |
| 互操作    | W3C 开放标准                     | 专有 SDK/API 生态                             |

三个关键差异:① Palantir 的革命性在"动词"——Action 让本体从"世界的地图"变成"行动的操作系统";② 形式主义换实用主义——牺牲形式推理,换取事务性写回与业务绑定;③ "受治理的动态本体"(变更即决策)精神上其实更接近 Palantir 而非 W3C。

Semantica 恰好站在两条腿上

Semantica(README 自称 "The Open Source Palantir for AI Agents")同时持有两套:Palantir 式决策层(决策节点/审批链/因果链/溯源)+ W3C 全栈 (RDF/OWL/SHACL/SPARQL——Palantir 没有的)。双轨兼有,是它敢这么自称的底气。

2 背景二:PDF 是 Palantir 的组织学翻译

Palantir 用工程语言(类型/管道/API)写同一结构,PDF 用制度语言(责任/采纳/追责/账本)重写。同一副骨架,推导方向相反:Palantir 自底向上(数据→语义→行动),PDF 自顶向下(制度公理→必要构件)。这就是"组织学翻译"的精确含义。

PDF 逐概念对照 Palantir

| PDF 原文概念                  | Palantir 对应物       | 翻译了什么                                   |
|---------------------------|--------------------|-----------------------------------------|
| 业务对象"决策节点对其写入状态"          | Object 属性写回        | 写回被译成制度动作——"状态写入是决策成立的关键制度动作"           |
| 决策节点五要素(目标/输入/依据/输出/责任主体) | Action 类型签名        | 几乎逐字段转写;PDF 多出 Palantir 隐含不表的责任主体必填     |
| 知识单元"只读、带版本、必须显式引用"       | 校验规则 / Function    | Palantir 把规则当功能;PDF 把"引用规则"升格为审计通过的前提   |
| 强制 HITL"事前结构化定义,运行中不可绕过"  | Action 审批链         | 从"功能"升格为"结构性约束"——责任不得隐性转移               |
| 决策账本"追加写入,不得修改"           | Action 审计日志        | PDF 字段清单几乎是审计记录的规范化超集+显式 append-only    |
| 决策图谱+轨迹"沿路径串联,支持回放"       | 图 + action lineage | 一边叫 lineage,一边叫轨迹——都是"可回放的因果谱系"         |
| Skill"可替换性与可审计性,而非智能程度"   | Functions + AIP 工具 | 一句话总结 AIP 架构立场:agent 可提议,采纳必走 Action+审批 |
| 黑箱对用户/白箱可审计               | 权限标记体系             | 同一原则的两种表述                               |
| 第一性约束(可审计+可追责)            | 对监管行业的整个卖点         | Palantir 用工程实现;PDF 写成公理                 |

PDF 不只是翻译——三处走得更远:① 失效模式(跨轨迹归纳结构性失效→反哺机制修订,组织学习的一等概念,Palantir 无对应);② 入口决策(对"是否启动决策路径"设闸);③ 判断/采纳/决策三分(比 Action 模型更精细地回答"LLM 输出什么时候才算数")。

## 同构 构件关系语法的咬合

决策节点 —读取→ 业务对象(事实/证据)   △  Action 读取 Object 属性  
决策节点 —引用→ 知识单元(版本化规则)   △  Action 执行校验逻辑(带版本)  
决策节点 —调用→ Skill(可替换能力)   △  Action 调用 Function/工具  
决策节点 —写入→ 业务对象状态   △  Action 的 transactional writeback  
决策节点 —生成→ 决策账本(不可变记录)   △  Action 审计落账  
失效模式 —反馈至→ 图谱/知识单元修订   △  (无直接对应—PDF 独有)

连 PDF 明言"构件关系不描述实现方式",都对应 Palantir"语义层与底层管道解耦"的立场。

### 3 核心概念:Action — 本体世界的动词

一句话:读世界靠查询,改世界必须走 **Action**。Action 是本体中唯一被治理的写回通道——带类型、带校验、带审批、带审计的事务性写操作。提交本身就是对象,全程留痕。

#### Palantir 四个本质属性 + 生命周期

| 属性  | 含义                               |
|-----|----------------------------------|
| 类型化 | 像 API 签名:参数、目标对象、输出都是声明好的类型      |
| 校验  | 提交时跑规则:类型检查 + 逻辑规则(可查本体其他对象做依据)  |
| 审批  | 可配置审批链,按风险分级——低风险自动放行,高风险强制人工    |
| 审计  | 每次提交/审批/执行都落账:谁、何时、什么参数、谁批的、改了什么 |



边界规则(不是所有操作都是 **Action**):读/算 = **Function**(不改状态,无需审批);写回 = **Action**(唯一通道,必经校验/审批/审计)。例:算 DTI 是 Function;改 creditLimit 是 Action。

## 示例：「提高信用额度」完整走查

载体是本次会议一直沿用的 A-7291(收入 85k、36 个月干净记录、风险 B2)。**橙注**标出制度语义, **蓝注**标出工程机制。

### 定义 Action 类型(事先声明)

#### Action: IncreaseCreditLimit

目标对象: Customer

参数:

customerId : Customer 链接(必填)  
newLimit : Decimal(必填)  
justification : String(必填)

校验规则:

R1 newLimit > 当前额度

△ 知识单元(版本化, 判断必须引用)

R2 newLimit ≤ 5 × 年收入

可查询对象属性做依据

R3 客户不在制裁名单(查另一组对象)

审批策略(按风险分级):

△ 风险阈值 → 强制 HITL

≤ \$10k → 自动放行

> \$10k → 风控官审批(单步)

> \$100k → 风控官 + 分行经理(两步)

执行效果(写回 + 连锁):

Customer.creditLimit = newLimit

△ 状态写入(决策成立的制度动作)

新增 LimitAuditEvent 对象并链接

△ 决策账本(append-only)

触发 Function: 重算信用率、通知、回写核心系统

△ ricochet 连锁

## 运行时 AIP agent 发起(A-7291,10 万美元)

- 1 **agent 提议** — LLM 分析客户对象 → 提交 Action(newLimit=100000 + justification)。提交者是 agent,但没有任何特权,走同一通道。△ 判断:对组织尚无约束力。
- 2 **校验** — R1 ✓ R2 ✓ R3 ✓。任一失败:submission 打回,带失败原因。
- 3 **风险路由** — 100k 触及两步审批阈值 → 不自动放行,进入审批流。
- 4 **人工审批** — 风控官看到:提议者(agent,标注)、justification、校验结果、客户上下文 → 可批准/驳回/改参。风控官 ✓ → 分行经理 ✓。△ 采纳:制度分界点,从此组织被绑定。
- 5 **执行写回** — creditLimit: 50k → 100k;LimitAuditEvent 落账(before/after、全员、时间戳);连锁:用信率重算、通知、核心系统同步。
- 6 **留痕** — submission 对象永久保存:agent 提议 → 校验 → 2 人审批 → 执行,整链可回放。△ 决策轨迹。

为什么不让 **agent** 直接改数据? ① 单一写通道 = 治理锚点,无旁路;② agent 被降权为"提议者"——责任始终落在授权的人;③ 风险分级是声明式配置,改阈值本身也是受治理的变更。

## 5 扩展:完整的 Action 生态

一个领域里的 Action 不是孤例,而是覆盖生命周期的**动词集合**——且审批策略有丰富度:分层(自动/单审/两步/冷却期),甚至有反向需求(欺诈标记要"快"而非"严")。

全景

贷款领域的 Action 清单(按生命周期)

| 阶段      | Action                | 风险/审批         | 写回什么                       |
|---------|-----------------------|---------------|----------------------------|
| 受理      | SubmitLoanApplication | 低·自动          | 创建 Application(状态=已受理)     |
|         | RejectAtIntake        | 中·单审          | Application.status=拒收      |
| 核验      | VerifyIncome          | 低·自动          | incomeVerified=true + 证据链接 |
|         | OrderCreditReport     | 低·自动          | 创建 CreditReport 对象         |
|         | FlagFraudSuspect      | 中·快速通道        | riskFlag=可疑(要快,不能等两步)      |
| 决策      | ApproveLoan           | 高·两步          | Loan.status=已批准            |
|         | RejectLoan            | 高·单审          | status=已拒绝(拒也要留理由——公平性审计)  |
|         | WaivePolicy           | 最高·双人+理由必填    | 创建 PolicyException 对象      |
|         | IncreaseCreditLimit   | 分级            | Customer.creditLimit       |
|         | FreezeCredit          | 中·单审          | frozen=true                |
| 定价签约    | SetInterestRate       | 中·单审          | Loan.rate=8.9%             |
|         | DisburseLoan          | 最高·不可逆·双人+冷却期 | status=已放款(资金动账)           |
| 贷后      | RecordPayment         | 低·自动(对账)      | balance↓                   |
|         | FlagMissedPayment     | 低·自动          | delinquency=1              |
|         | RestructureLoan       | 高·两审          | 条款对象重建                     |
|         | UpdateRiskTier        | 中·单审          | riskTier=B2→B1(影响后续所有定价)   |
| 治理(元行动) | RevisePolicy          | 最高·变更委员会      | 知识单元版本号+1                  |

### 三类特殊 Action 性质不同的动词

#### ① 入口决策型 · SubmitLoanApplication

决定一条决策路径是否被激活——被拒收,后续所有 Action 都不发生。△ PDF"入口决策":判断对象不是业务结论,而是路径是否应当启动。

#### ② 不可逆型 · DisburseLoan

前置状态机校验 → 双人审批 + 4 小时冷却期(可撤销窗口) → 放款后审计永不删改。不可逆动作享受最高规格——资金动过的事实必须永远可回放(账本 append-only 的制度理由)。

#### ③ 元行动型 · RevisePolicy

改的不是业务对象,是规则本身:新旧差异报告 + 受影响存量决策清单 → 变更委员会审批 → 版本+1、旧版归档保留。未来校验用新版,历史决策仍引用当时的旧版——可回放的关键。△ 立法行动;接"受治理的动态本体"(变更即决策)。

回看因果链:会话最初的 `credit_application → loan_underwriting → interest_rate` 因果链,Action 视角下即 `SubmitLoanApplication → ApproveLoan → SetInterestRate` 的轨迹——PDF 的"决策轨迹"△ Action 序列的回放。

## 6 灵魂:PDF 的制度观(五命题)

制度观 = 把设计的出发点从"系统如何运转"换成"组织何时受一项行为的约束"。工程观问 *how it works*,制度观问 *when it binds*。五个命题均出自 PDF 原文。

## 一 决策是制度性结果,不是技术操作

"决策是**被组织采纳**、写入业务对象状态并**承担责任**的制度性结果。" "采纳是判断与决策之间的**制度分界点**。"

三分:**判断**(人或 Agent 产生,无制度后果)→ **采纳**(组织受约束的时刻)→ **决策**。写一条数据库记录 ≠ 组织做了一个决定。

## 二 公理是制度的,不是技术的

第一性约束 = 可审计 + 可追责——面向监管与责任分配的**制度前提**,不满足则组织**无权采纳**。全部构件由这条公理推出。

## 三 责任先于功能

"任何无法明确责任主体的判断,**不构成**决策节点。" "防止责任在自动化判断过程中被**隐性转移**。"

责任主体必填、授权边界事前冻结、HITL 不可绕过——"这步是自动还是人工"在制度观里是**责任归属问题**,不是实现细节。

## 四 记录具有制度效力(不只是数据)

"账本记录一经生成,**不得被修改或覆盖**,仅允许追加。"

账本不是 log(工程物),是**不可抵赖的制度凭证**——公证处的登记簿,不是数据库的 binlog。回放服务于审计与追责,不是调试。

## 五 演进本身受治理

可演进对象:风险阈值、HITL 触发条件、路径结构、知识单元内容;但"**第一性约束本身不可演进**"——公理不动。失效模式(跨轨迹归纳结构性失效→反哺修订)= 组织学习的制度化。等价于**宪制思维**:条文可修,修法本身有程序。

对照 工程观 vs 制度观 —— 示例里"注入"了什么

| 问题              | 工程观(Palantir 语言)     | 制度观(PDF 语言)          |
|-----------------|----------------------|----------------------|
| 一次写回是什么         | Action 执行:校验通过、事务写库  | 采纳:授权的人在授权边界内让组织受到约束 |
| agent 的输出       | 一次 API 调用的结果         | 判断——采纳之前对组织无约束力      |
| 为什么审计           | 可观测性功能               | 不可抵赖性——追责的制度前提       |
| 为什么 append-only | 存储层不可变               | 记录是制度凭证,涂改即失信        |
| 规则是什么           | validation logic(代码) | 知识单元——版本化、必须引用的"法条"  |
| 改规则             | 发版部署                 | 立法行动——本身要过治理         |
| 自动化             | 效率特性                 | 责任的转移方式——必须显式且经授权    |

注入映射:agent 填参提交=判断 / 风控官点批准=采纳 / 校验规则 R1-R3=知识单元 / LimitAuditEvent=决策账本 / 放款冷却期+永不删审计=不可逆制度事实 / RevisePolicy=立法。没有这层注入,Action 只是"带审批流的 CRUD";注入之后,它回答的才是监管真正的问题——"凭什么说这是组织的决定?谁担责?凭什么信这份记录?"

7 现实边界:哪些是项目已有的,哪些是新设计的

诚实声明:示例中的 Action 全部是为讨论新设计的——项目里既没有"Action"抽象,也没有其中任何一个具体 Action。项目已有的是"影子"。

血统 示例的设计来源构成



载体:Semantica 真实数据(A-7291 + 已有数据类)。另两点声明:① 基于 Palantir 公开文档,非其内部实现复刻;② 非 W3C/OWL 建模(类型化对象思维本身即 Palantir 对 W3C 的偏离)。

| 设计元素                                                          | 来源                  |
|---------------------------------------------------------------|---------------------|
| Objects / Links / Function-Action 边界 / 生命周期 / 风险分级 / ricochet | Palantir 公开模型(≈70%) |
| 入口决策标注 · 知识单元=版本化校验规则 · 判断/采纳 · 不可变账本                         | 《决策原生方法论》(≈20%)     |
| 元行动(RevisePolicy / ApproveOntologyChange = 本体变更本身是 Action)    | 本讨论的合成(≈10%)        |

盘点 Semantica 已有的"影子"(会话中核实)

| 项目已有                                | 位置                 | 与 Action 的差距             |
|-------------------------------------|--------------------|--------------------------|
| ApprovalChain (审批人/方式/上下文)          | decision_models.py | 记录审批发生过,不阻断未审批流程         |
| PolicyException (豁免+审批人+理由)         | decision_models.py | 记录豁免,无"双人批准"强制           |
| Policy.version                      | decision_models.py | 有版本号,无"修订是受治理变更"机制       |
| check_decision_rules / PolicyEngine | policy_engine.py   | 事后检查返回 verdict——非事前校验+阻断 |
| record_decision + PROV-O            | context_graph.py   | 记录决策,但决策不产生"写回状态"的效果     |

一句话概括差距:项目有"决策与审批的账本"(记录发生了什么),没有"Action引擎"(规定什么被允许发生、不批就不发生)。前者是史学,后者是法治。

8 结论:对 fork 的意义

## 收束 三句话

- 一 **Palantir 给了 Action 一副骨架(它怎么跑),PDF 给了 Action 一个魂(它凭什么算数)**。骨架解决"单通道写回可治理",魂解决"组织何时被绑定、谁担责、凭证何以可信"。
- 二 **Semantica 已是这份规格的开源半实现:决策节点/业务对象/账本/轨迹** ;阻断式 HITL  (记录非阻断);append-only 账本 ;失效模式 。差的恰是"从工程走向制度"的最后一步。
- 三 **自然的下一步:在 fork 里实现阻断式 Action 引擎原型——校验规则=知识单元版本、风险分级路由审批、不批不写回、全链入账**。那将一次补齐 PDF 的三个差距(强制 HITL / 状态写入 / 知识单元版本化),也是"开源版 Palantir"从口号变成代码的一步。本文档即其设计依据。

文档说明 — 整理自围绕贷款示例(A-7291)的讨论:两种本体论之辨 → PDF 与 Palantir 的同构 → Action 概念与示例 → Action 生态 → 制度观五命题 → 已有/新设边界 → fork 路线。

事实性内容(项目已有的类、方法、行为)均经源码核实;Palantir 部分基于公开文档概念;《决策原生方法论》引文出自 [book/《决策原生方法论》.pdf](#)。

关联文件: [examples/case/01\\_decision\\_intelligence/](#) (决策链演示)· [book/Agno双Agent决策序列图.html](#) (agent 集成视角)· [book/决策图谱节点信息.html](#) (图存储视角)。